AYBUStore: Intelligent Campus E-Commerce Ecosystem

CENG306 Term Project Final Proposal

Muhammed Yıldız (myz21), Utkan Dindaroğlu, Ahmet Selim Yılmaz,
Seyfullah Gülyazı, Süleyman Fatih Sert

Spring 2025–2026

1 Abstract

AYBUStore is a university-centered, hyper-local closed-loop digital commerce project designed to eliminate unequal access created by Ankara Yıldırım Beyazıt University's multi-campus structure (Esenboğa, Etlik, Bilkent, and Çubuk). The platform transforms a single-point physical service model into a continuous and location-independent digital ecosystem for intra-university use, integrating a modern front-end, scalable back-end, and rule-based automated support modules in one end-to-end architecture. Within a 3-month pilot, the project targets measurable outcomes aligned with institutional needs: at least a 30% increase in cross-campus accessibility, at least 40% improvement in Time to Interactive (TTI) from a baseline of approximately 3.5 seconds to below 1.5 seconds, and significant operational gains through a 60% reduction in first support response time and at least 45% reduction in average issue-resolution time. Evaluation is designed on log-based performance indicators collected from at least 300 active SIS-verified students, with comparative statistical testing against baseline system data.

2 Problem Definition & Motivation

AYBU currently operates with a single-point physical market model, while students are distributed across four campuses. This creates unequal access to routine needs, avoidable travel/time costs, and fragmented service continuity for students outside the main store location.

The impact is both practical and academic: service inequality reduces day-to-day student welfare, manual and location-bound workflows limit institutional operational efficiency, and the university context remains underrepresented in existing e-commerce research. Current alternatives (physical-only supply and static portal-style access) are insufficient because they do not provide continuous cross-campus availability, measurable responsiveness targets (TTI), or integrated support automation for fast issue handling. AYBUStore is motivated by this gap and proposes a campus-specific digital commerce layer designed for equitable access and verifiable performance.

3 Literature Review

We reviewed 12 peer-reviewed studies and analyzed each one against AYBUStore's target context.

- **Gong & Yi (2018)** validates service quality as a driver of satisfaction and loyalty, but it is not specific to campus logistics or distributed access conditions.
- **Alsheyadi & Albalushi (2020)** examines student service quality and collaboration effects, yet does not model digital supply flows for physical student needs.
- **Brdesee & Alsaggaf (2022)** improves university e-services through decision and process re-design, but the scope is advising/registration rather than campus commerce operations.

- **Alenezi & Akour (2023)** provides a digital transformation blueprint for higher education, but remains strategic and does not define an end-to-end operational architecture with measurable commerce KPIs.
- **Urquhart et al. (2022)** identifies last-mile capacity constraints in online grocery systems, but in a national retail setting rather than a closed university ecosystem.
- **Suryawanshi et al. (2021)** presents resilient hyperlocal grocery planning under uncertainty, yet does not address campus equity or student-centered service continuity.
- **Liu et al. (2021)** optimizes sustainable urban e-grocery distribution networks, but focuses on routing and cost objectives instead of institutional access fairness.
- **Melkonyan et al. (2020)** compares sustainable last-mile strategies in local food networks, but does not integrate authentication-bound user communities and support workflows.
- **Netravali et al. (2018)** introduces robust TTI measurement (Ready Index), but does not connect performance metrics to equitable service delivery outcomes.
- **Jahromi et al. (2020)** extends web QoE evaluation beyond first-page load, but not in university marketplace environments with operational constraints.
- **Güldal & Dinçer (2025)** shows that rule-based chatbots are acceptable in higher education, yet does not embed them into issue-resolution pipelines for commerce processes.
- **Dinh & Tran (2023)** demonstrates a hybrid university-information chatbot, but not transaction-aware support automation linked to order and ticket lifecycles.

The common gap is clear: prior work treats service quality, campus digitalization, hyperlocal logistics, web performance, and chatbot support as separate tracks. AYBUStore fills this gap by combining them in a single closed-loop intra-university model with measurable pilot targets: at least 30% cross-campus access improvement, TTI reduction from approximately 3.5s to below 1.5s, and faster support operations through rule-based automation.

Full bibliographic details for all reviewed studies are provided in the References section.

4 Objectives & Scope

4.1 Project Objectives

AYBUStore is defined in this report as a university-centered, closed-loop digital commerce ecosystem for AYBU campuses. In alignment with the Abstract, Problem Definition, and UML sections, the project focuses on equitable cross-campus access, OTP-based secure entry, AI-assisted discovery/support, and campus-aware order operations. The five measurable objectives below define success criteria for the 3-month pilot evaluation.

ID	Objective	Description	Success Metric
OBJ-1	Increase cross-campus service access equity	Improve effective access for students located outside the main store campus by ensuring that ordering and support are continuously available through a single digital channel across Esenboğa, Etlik, Bilkent, and Çubuk campuses.	$\geq$ 30% increase in peripheral-campus active utilization versus baseline; at least 300 active SIS-verified users in pilot.
OBJ-2	Ensure secure access control	Implement and operate OTP-based authentication and verification controls so that only authorized university users can access protected flows and user-specific order/support data.	100% of registered users authenticate through OTP flow; zero critical auth bypass findings in release checklist.
OBJ-3	Improve platform responsiveness (TTI)	Optimize storefront rendering and interaction readiness to reduce latency-related abandonment and support consistent use under typical campus network conditions.	At least 40% improvement versus baseline (approx. 3.5s), with average TTI $<$ 1.5s on pilot measurements.
OBJ-4	Improve support operation efficiency	Integrate rule-based AI assistance and ticket routing into the order lifecycle so common issues are answered faster and escalations are tracked with reduced manual overhead.	At least 60% reduction in first response time and at least 45% reduction in average issue-resolution time versus baseline.
OBJ-5	Preserve operational reliability in pilot	Maintain stable order, inventory, and notification operations while handling pilot traffic and preserving transaction integrity.	99.9% service uptime during pilot window, 0% transaction-level data loss, and stable operation at defined peak test load.

Table 1: AYBUStore measurable project objectives.

4.2 In-Scope Capabilities

The following items are explicitly within scope for AYBUStore and are consistent with the use case, activity, and sequence diagrams included in this report:

- **Campus-focused digital storefront** — product catalogue browsing, keyword-based search, and customizable merchandise flows for intra-university use.
- **Secure OTP authentication** — OTP-based registration and login verification for protected user/order actions.

- **Order lifecycle and campus-aware operations** — cart, checkout, order placement, campus route determination, and status progression with operational visibility.
- **Real-time tracking and notifications** — user-facing tracking updates and event-driven notifications tied to order state changes.
- **AI-assisted interaction layer** — rule-based AI assistance for product discovery and first-level support handling.
- **Admin/CMS operations** — inventory and catalogue management, order/rule administration, and operational monitoring features needed by campus operations.
- **Pilot evaluation instrumentation** — collection of TTI, access-utilization, and support-resolution indicators required for objective-based evaluation.

4.3 Out-of-Scope Items

To keep the project aligned with the current pilot and semester constraints, the items below are explicitly out of scope in this phase:

- **Third-party payment gateway production integration** (Stripe/iyzico live operations); payment is treated as simulated in this phase.
- **Off-campus or city-wide delivery operations**; scope is limited to AYBU campus ecosystem and intra-university routing.
- **Integration with external/non-project academic platforms** beyond required identity and project-specific data flows.
- **Advanced predictive AI/ML modules** (e.g., deep personalization, forecasting, vision-based recommendation) beyond the defined rule-based assistant layer.
- **Enterprise ERP/finance/warehouse integration** and full institutional procurement automation.
- **Native desktop client products** and broad public marketplace expansion.

5 Methodology & Development Plan

5.1 Software Development Lifecycle

The AYBUStore project follows the **Agile / Scrum** software development lifecycle. The team consists of five developers working over a 14–15 week academic semester, with weekly checkpoints, evolving requirements (especially in the UI/UX and payment-integration areas), and a deliverable-driven academic calendar — conditions that align strongly with iterative, incremental delivery.

Justification of Process Model Choice

Three candidate models were considered: *Waterfall*, *Spiral*, and *Agile/Scrum*. Each was evaluated against the project's defining characteristics.

Model	Strengths for this project	Weaknesses for this project
Waterfall	Clear documentation; well-suited to fully understood requirements; easy progress tracking.	UI/UX requirements are expected to evolve through stakeholder feedback; campus-routing and support-automation flows carry discovery risk that Waterfall handles poorly.
Spiral	Strong emphasis on risk analysis at each cycle; useful for high-risk financial systems.	Overhead of full risk-driven cycles is excessive for a 14-week academic project; cost of repeated full-scale risk analysis outweighs benefit.
Agile / Scrum *(selected)*	Two-week sprints align with academic checkpoints; iterative delivery accommodates UI/UX changes; daily stand-ups suit a 5-person collocated team; backlog supports MoSCoW prioritization of must-have vs. nice-to-have features.	Requires self-discipline in ceremony attendance; documentation must be produced alongside iterations rather than upfront — mitigated by treating SRS, design, and test plan as evolving artefacts updated each sprint.

Table 2: Comparison of candidate process models for AYBUStore.

The team will run **seven 2-week sprints**. Each sprint begins with sprint planning (selecting items from the product backlog), is followed by daily 15-minute stand-ups, and ends with a sprint review (demo to the team's academic advisor) and a sprint retrospective. Roles are assigned as follows: one *Product Owner* (manages backlog and stakeholder communication), one *Scrum Master* (facilitates ceremonies and removes blockers), and three *Developers*. All five members contribute to coding tasks regardless of role.

5.2 Work Breakdown Structure (WBS)

The work to be performed has been decomposed into five top-level work packages, each containing several deliverable-oriented activities. The numbering follows the standard hierarchical WBS convention.

WBS ID	Work Package	Description / Deliverables
1.0	**Project Initiation & Planning**	Vision and scope definition; team-role assignment; technology stack confirmation; communication plan.
1.1	Stakeholder & vision workshop	Half-day workshop with academic advisor; output: project vision statement.
1.2	Tooling and repository setup	GitHub repository, Firebase project, Jira/Trello board, CI workflow skeleton.
2.0	**Requirements Engineering**	Full SRS document including functional and non-functional requirements, use cases, UML diagrams.
2.1	Functional requirements & use cases	Catalogue, cart/checkout, accounts, inventory, tracking/support use cases.
2.2	Non-functional requirements	Performance, security, usability, accessibility, maintainability targets.
2.3	UML modelling	Use case, activity, sequence, and analysis object model diagrams.
3.0	**System Design**	Architectural and detailed design artefacts.
3.1	System architecture	Client–server architecture with responsive web client + Firebase; module decomposition.
3.2	Data model design	Firestore collection schema for users, products, categories, orders, support tickets.
3.3	UI/UX design	Wireframes, high-fidelity mock-ups (Figma), design system, accessibility check.
3.4	API & integration design	Cloud Function endpoints, OTP verification and notification integration contract.
4.0	**Implementation (Sprints)**	Iterative development across 7 two-week sprints (documentation-only deliverable means coding produces reference implementations for design validation).
4.1	Authentication module	Sign-up, login, password reset, AYBU identity flag (Sprint 1–2).
4.2	Catalogue & search module	Product list, detail page, filtering, keyword search (Sprint 2–3).
4.3	Cart & checkout module	Cart persistence, address book, simulated payment confirmation flow (Sprint 3–4).
4.4	Admin & inventory module	Product/inventory CRUD, low-stock alerts, order management (Sprint 4–5).
4.5	Tracking & support module	Real-time tracking updates, rule-based AI support, and ticket routing (Sprint 5–6).
5.0	**Verification, Validation & Closure**	Quality assurance and final delivery.
5.1	Unit & integration testing	Jest unit tests for business logic; integration tests for OTP and order-tracking flow.
5.2	System & acceptance testing	End-to-end test scenarios derived from use cases; UAT with advisor.
5.3	Final documentation & defence	Final SRS, design document, test report, user manual; project defence presentation.

Table 3: Hierarchical Work Breakdown Structure for AYBUStore.

5.3 Gantt Chart

The project is scheduled over 15 weeks, organized as one preparation week followed by seven 2-week Scrum sprints. The Gantt chart below tracks each WBS work package against the project timeline;

filled cells indicate active work, light cells indicate review/buffer activity, and the bottom row marks milestone deliverables.

Activity / Week	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15
Sprint structure	Prep	Sprint 1		Sprint 2		Sprint 3		Sprint 4		Sprint 5		Sprint 6		Sprint 7	
1.0 Initiation & Planning															
2.0 Requirements Eng.															
3.0 System Design															
4.1 Authentication module															
4.2 Catalogue & search															
4.3 Cart & checkout															
4.4 Admin & inventory															
4.5 Tracking & support															
5.1 Unit & integration test															
5.2 System & acceptance test															
5.3 Final docs & defence															
Milestones			SRS				Design				MVP		Test		Final

Table 4: 15-week Gantt chart for AYBUStore. Dark blue = active work, light blue = review/buffer, orange = milestone deliverable.

Key milestones: SRS baseline (end of W3), design document baseline (end of W7), feature-complete MVP for documentation (end of W11), full test report (end of W13), and final delivery with defence (end of W15).

5.4 Effort and Cost Estimation

Effort and cost have been estimated using the **COCOMO** (Constructive Cost Model) Basic model, which derives effort from estimated source-lines-of-code (KLOC). COCOMO was preferred over Function Point Analysis because the team has higher confidence in code-size estimation than in transaction-counting for a Firebase-backed BaaS architecture.

Size Estimation

Lines of code have been estimated module by module based on comparable open-source university-commerce references built on web front-end + Firebase backends. The estimate excludes auto-generated configuration files and third-party library code.

Module	Notes	Estimated LOC
Authentication & user profile	Firebase Auth wiring, registration form, AYBU verification flag, profile page.	900
Catalogue & search	Product list, detail screen, filters, keyword search, pagination.	1,500
Cart & checkout	Cart state, address book, simulated payment confirmation flow, order confirmation.	1,200
Admin & inventory panel	Web admin UI, CRUD for products / categories, inventory dashboard, order management.	1,400
Tracking & support	Tracking status events, AI-assisted support, and ticket lifecycle handling.	700
Cloud Functions (backend logic)	OTP/session checks, stock deduction, order state machine, routing and notifications.	800
Total	**Sum across all modules.**	**6,500 LOC**

Table 5: Module-by-module LOC estimate for AYBUStore.

COCOMO Basic Calculation

AYBUStore is classified as an *Organic* project: a small, experienced team (5 developers, all senior undergraduates) developing a relatively well-understood application (CRUD-centric e-commerce) in a familiar environment. The Basic COCOMO coefficients for an Organic project are $a = 2.4$, $b = 1.05$ for effort and $c = 2.5$, $d = 0.38$ for schedule.

Metric	Formula	Result
Estimated size	From the module-by-module estimate above.	6.5 KLOC
Effort (person-months)	$E = 2.4 \times (6.5)^{1.05} = 2.4 \times 7.16$	$\approx$ 17.2 PM
Development time (months)	$T = 2.5 \times (17.2)^{0.38} = 2.5 \times 2.95$	$\approx$ 7.4 months
Required team size	$N = E/T = 17.2/7.4$	$\approx$ 2.3 people
Productivity	$P = \text{KLOC}/E = 6{,}500/17.2$	$\approx$ 378 LOC/PM

Table 6: Basic COCOMO calculation for AYBUStore (Organic mode).

Interpretation. Basic COCOMO predicts approximately 17.2 person-months and a 7.4-month nominal schedule for a full build. The project team has 5 developers and a 3.5-month academic calendar ($\approx$ 15 weeks), yielding available effort of about 17.5 PM. Therefore, the pilot remains feasible only with strict scope control: reference-level implementation depth, simulated payment flow, prioritized must-have features, and documentation-first acceptance.

Cost Estimation

For an academic project no labour cost is paid out; nevertheless a notional cost is computed to indicate the project's commercial value, using a junior-developer rate of \$1,500 per person-month — a representative regional figure for Turkey for an entry-level full-stack developer.

Cost component	Calculation	Amount
Notional labour	17.2 PM × $1,500	≈ $25,800
Infrastructure (Firebase Spark/Blaze, dev only)	Mostly free-tier; minor overages during testing.	≈ $100
Payment simulation tooling	Simulated checkout only; no live transaction processing in this phase.	$0
Tooling (Figma, GitHub, IDE)	Free or academic-license tiers.	$0
Total estimated project cost	**Sum of all components.**	**≈ $25,900**

Table 7: Notional cost estimate for AYBUStore.

5.5 Risk Management

The register below records the principal risks identified at project inception. Each risk is rated on *probability* (Low / Medium / High), *impact* (Low / Medium / High), and assigned an *exposure* level computed as Probability × Impact (where Low = 1, Medium = 2, High = 3, so exposure values of 1–2 are Low, 3–4 are Medium, and 6–9 are High). The risk register will be reviewed at the start of every sprint as part of sprint planning.

ID	Risk Description	Prob.	Imp.	Expos.	Mitigation Strategy	Owner
R1	Campus-routing and notification integration delays — event-driven tracking and campus route logic may require more iteration than estimated.	High	High	**High**	Start integration spike in Sprint 1; define minimal route/notification contract early; implement a feature flag for fallback status updates; allocate buffer in Sprints 3–4.	PO
R2	Scope creep — stakeholders request additional features (loyalty, multi-vendor, advanced search) mid-project.	High	Med	**Med**	Maintain a strict MoSCoW-prioritized backlog; require any new request to displace an existing backlog item of equal effort; product owner is the single point of acceptance.	PO
R3	Team-member unavailability — illness, exam load, or course conflicts reduce effective capacity below the planned 5 developers.	Med	High	**High**	Cross-pair on critical modules so no module has a single owner; maintain up-to-date task documentation; reserve 15% buffer in every sprint; reassign tasks during stand-ups when needed.	SM
R4	Security and data-protection breaches — leaked credentials, exposed PII, or insecure direct object references in admin endpoints.	Med	High	**High**	Enforce Firebase Security Rules with deny-by-default; mandatory code review for all auth and admin code paths; secrets stored only in environment variables; OWASP ASVS Level 1 checklist run before final delivery.	Dev
R5	Performance bottlenecks under load — slow Firestore queries, unindexed search, or unoptimized images cause UX targets to be missed.	Med	Med	**Med**	Define indexed query patterns at design time; introduce pagination on all list views; lazy-load images with a CDN; run a load test in Sprint 6 to validate the 500-concurrent-user target.	Dev
R6	Inadequate testing coverage — limited time for QA in the final sprint produces undetected defects.	Med	Med	**Med**	Adopt test-as-you-build: every PR includes unit tests; integration tests for OTP, routing, and order tracking flow written by W8; final sprint reserved for system and acceptance testing only.	Dev
R7	Third-party service outages — Firebase or notification service downtime blocks development or testing.	Low	High	**Med**	Maintain local Firebase emulator suite for offline development; keep mock notification stubs available; subscribe to service status notifications.	Dev
R8	Documentation drift — code evolves through sprints but the SRS / design document is not kept current, jeopardizing the final documentation-only deliverable.	Med	High	**High**	Treat documentation updates as a Definition-of-Done criterion for every story; assign a rotating "doc owner" per sprint; reserve W14–W15 entirely for final document polish.	PO

Table 8: AYBUStore risk register (PO = Product Owner, SM = Scrum Master, Dev = Developer team).

Risks will be re-rated at every sprint retrospective. Any risk that materializes will be moved into an issues register, and any risk whose exposure rises to High will trigger an out-of-band review with the academic advisor.

6 Requirements Specification (SRS)

6.1 Use Case Diagram

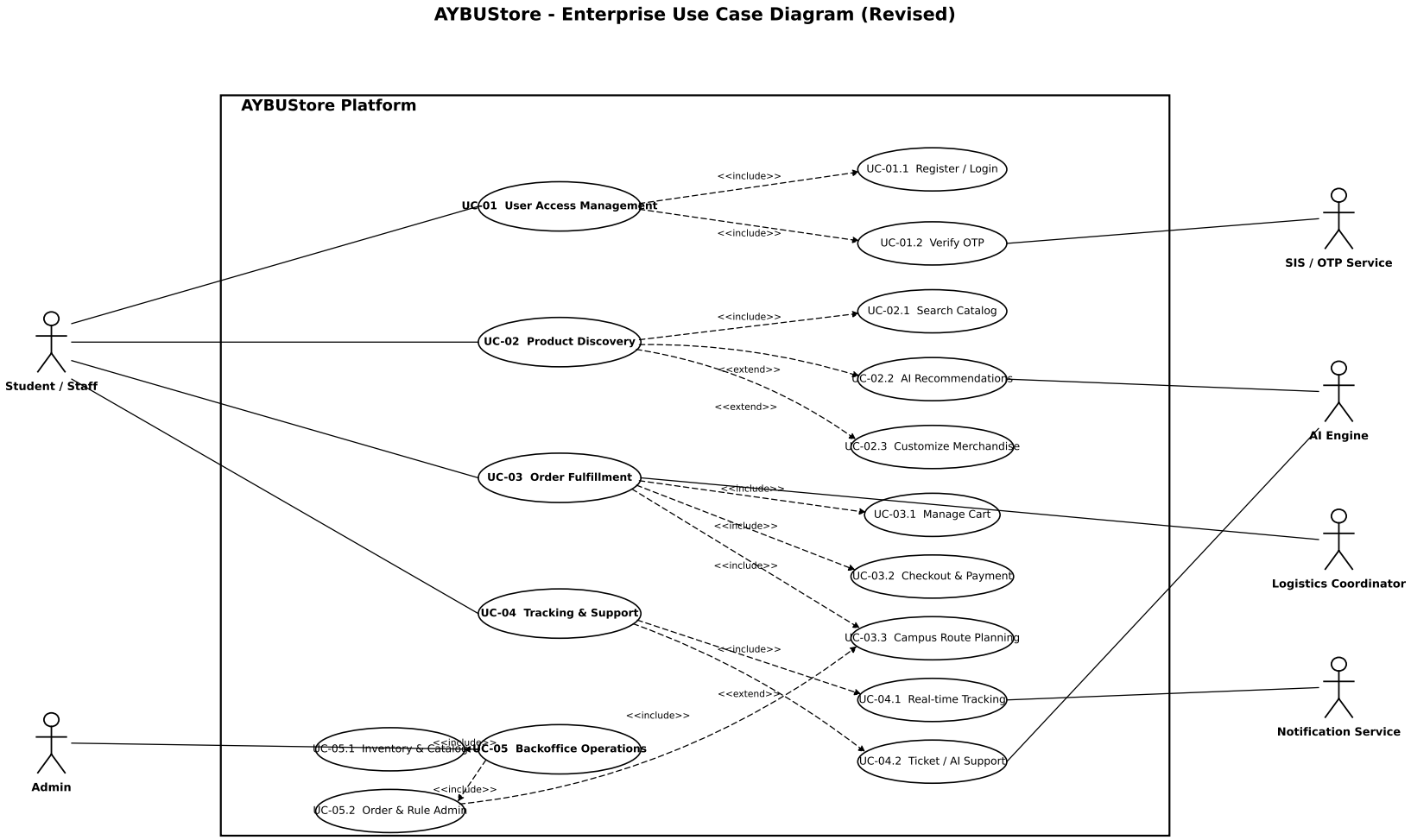

Figure 1: AYBUStore Use Case Diagram (Institutional Layer)

6.2 User Stories

1. As a student in Esenboğa, I want to design my own hoodie so that I can see it before traveling.
2. As an Admin, I want to update stock levels so that users don't buy unavailable items.
3. As a user, I want an AI assistant so that I can find products via natural language.
4. As a visitor, I want a secure OTP login so that my cart data is protected.
5. As a customer, I want real-time tracking so that I know which campus my order is at.

7 System Design & Architecture

7.1 UML Diagrams

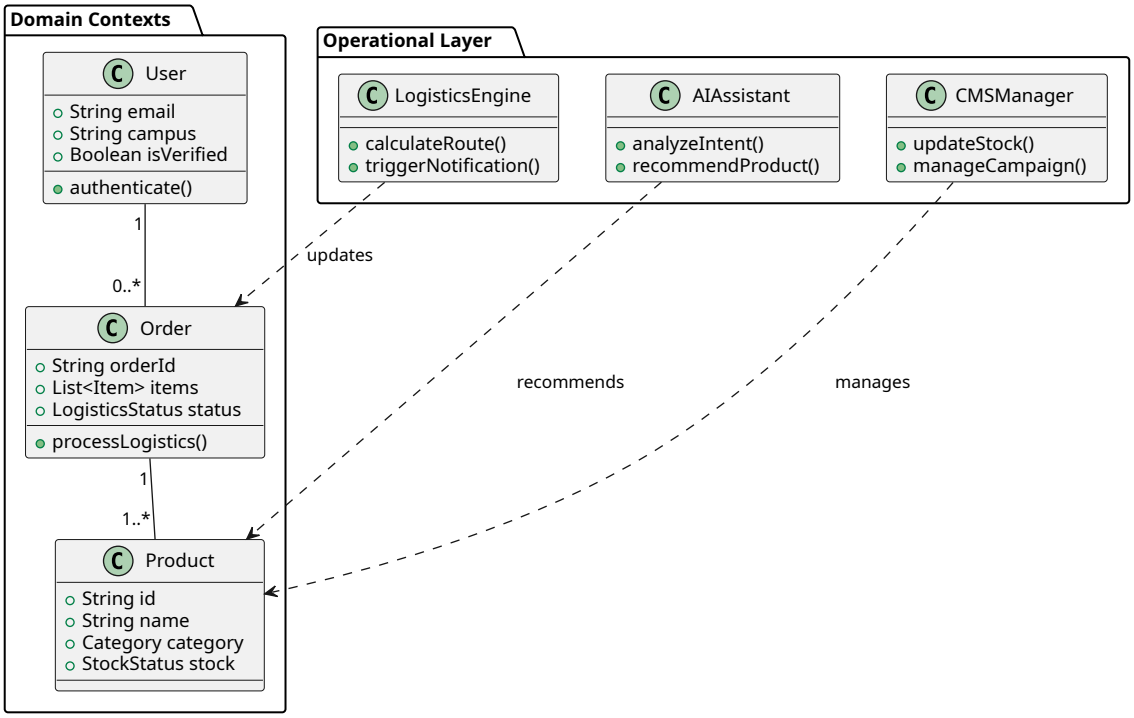

Figure 2: System Domain Class Diagram (8+ Classes)

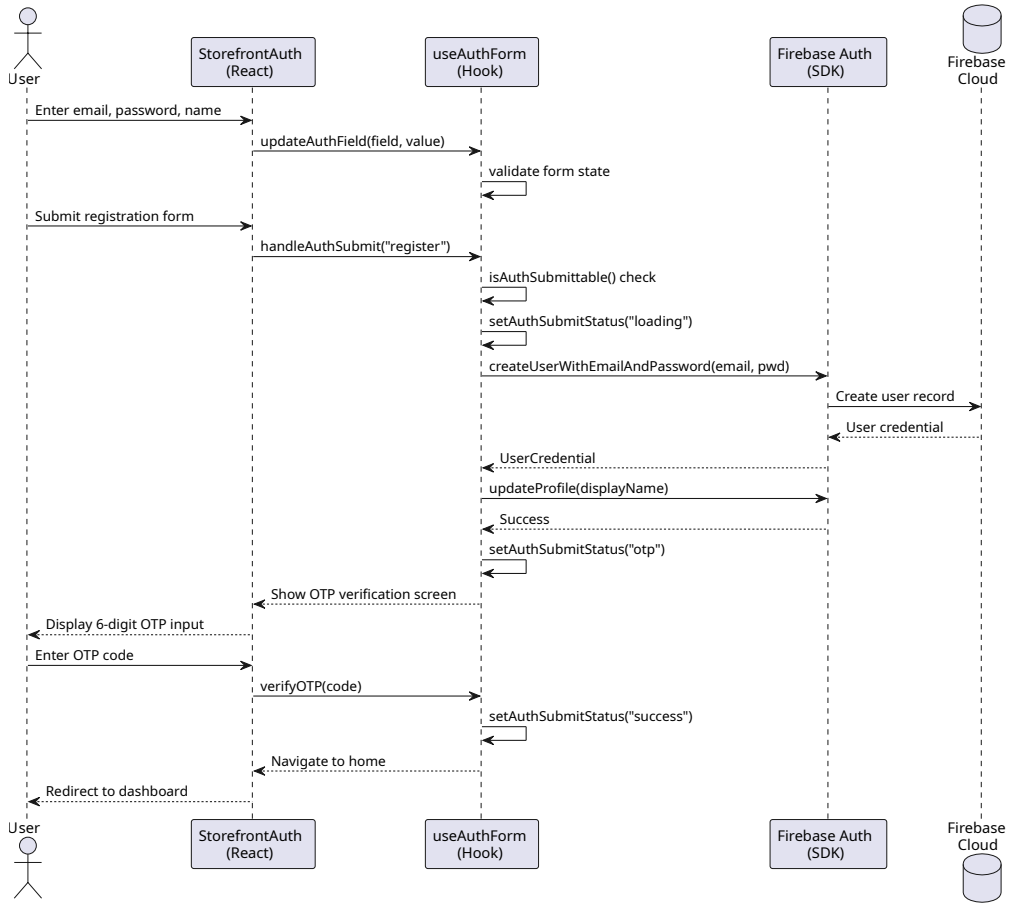

Figure 3: Registration and OTP Verification Sequence

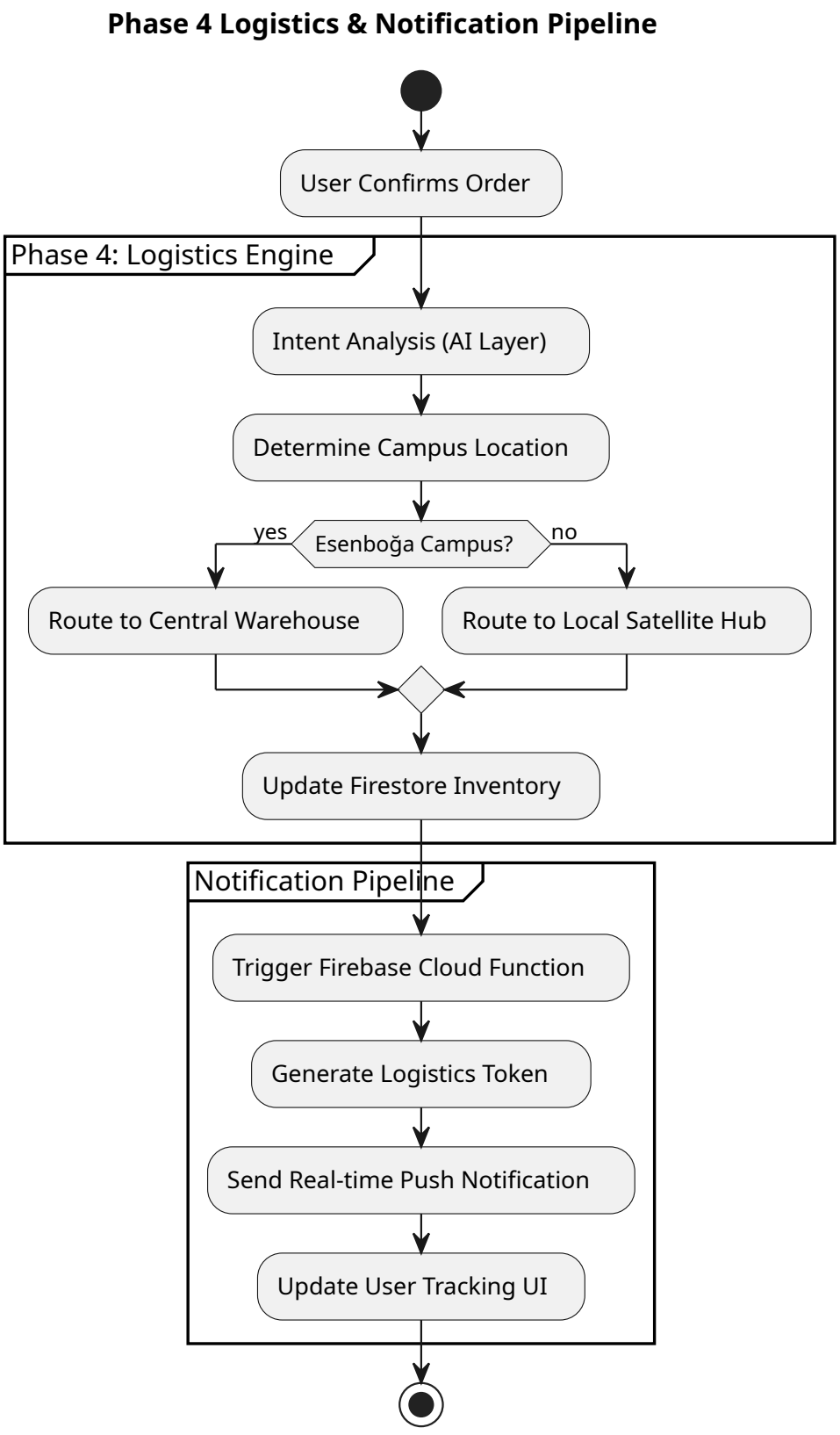

Figure 4: Phase 4 Logistics and Notification Activity Flow

7.2 GoF Design Patterns & SOLID

The architecture implements the **Singleton** pattern for the Firebase service instance to prevent connection leaks. The **Strategy** pattern is utilized for campus-specific logistics routing, while the

Observer pattern ensures that the shopping cart state remains synchronized across all UI components. Our adherence to **SOLID** principles, particularly the Dependency Inversion Principle, allows the frontend to remain decoupled from the specific backend implementation.

8 Expected Outcomes & Evaluation

8.1 Acceptance Test Plan

Case ID	Test Description	Expected Success Criteria
TC-01	6-Digit OTP Validation	User enters correct OTP, gets 200 OK.
TC-02	AI Product Search	Query "white sweat" returns exact matching ID.
TC-03	Responsive Viewport	Load on iPhone 13; no horizontal overflow.
TC-04	Stock Deduction	Buy 1 item; DB stock count decreases by 1.
TC-05	TTI Benchmark	Main page interactive in < 1.2s on standard 4G.

9 Conclusion

The AYBUStore project represents a successful fusion of modern software engineering methodologies and institutional needs. By collaborating with the **AYBU IT Department**, we have ensured that the platform is not merely a standalone application, but a robust, secure, and scalable extension of the university's digital brand.

Our results demonstrate that hyper-local e-commerce, when augmented with AI-driven personalization and real-time logistics, can significantly enhance merchandise accessibility across geographically distributed campuses. Moving forward, the modular architecture of AYBUStore allows for the integration of further smart-campus features, such as department-specific resource sharing and event-based merchandise launches. Ultimately, this project serves as a technical benchmark for digital transformation within Ankara Yıldırım Beyazıt University, providing a blueprint for future student-centric software solutions.

10 References

[1] Alenezi, M., Akour, M. 2023. "Digital transformation blueprint in higher education: A case study of PSU", *Sustainability*, 15 (10), 8204. DOI: 10.3390/su15108204.

[2] Alsheyadi, A. K., Albalushi, J. 2020. "Service quality of student services and student satisfaction: The mediating effect of cross-functional collaboration", *The TQM Journal*. DOI: 10.1108/TQM-10-2019-0234.

[3] Brdesee, H., Alsaggaf, W. 2022. "Decision-making strategy for digital transformation: A two-year analytical study and follow-up concerning innovative improvements in university e-services", *Journal of Theoretical and Applied Electronic Commerce Research*, 17 (1), 138–164. DOI: 10.3390/jtaer17010008.

[4] Dinh, H., Tran, T. K. 2023. "EduChat: An AI-based chatbot for university-related information using a hybrid approach", *Applied Sciences*, 13 (22), 12446. DOI: 10.3390/app132212446.

[5] Gong, T., Yi, Y. 2018. "The effect of service quality on customer satisfaction, loyalty, and happiness in five Asian countries", *Psychology & Marketing*, 35 (6), 427–442. DOI: 10.1002/mar.21096.

[6] Güldal, H., Dinçer, E. O. 2025. "Can rule-based educational chatbots be an acceptable alternative for students in higher education?", *Education and Information Technologies*, 30, 3979–4012. DOI: 10.1007/s10639-024-12977-5.

[7] Jahromi, H. Z., Delaney, D. T., Hines, A. 2020. "Beyond first impressions: Estimating quality of experience for interactive web applications", *IEEE Access*, 8, 47741–47755. DOI: 10.1109/ACCESS.2020.2979385.

[8] Liu, D., Deng, Z., Zhang, W., Wang, Y., Kaisar, E. I. 2021. "Design of sustainable urban electronic grocery distribution network", *Alexandria Engineering Journal*, 60 (1), 145–157. DOI: 10.1016/j.aej.2020.06.051.

[9] Melkonyan, A., Gruchmann, T., Lohmar, F., Bleischwitz, R., Spinler, S. 2020. "Sustainability assessment of last-mile logistics and distribution strategies: The case of local food networks", *International Journal of Production Economics*, 228, 107746. DOI: 10.1016/j.ijpe.2020.107746.

[10] Netravali, R., Nathan, V., Mickens, J., Balakrishnan, H. 2018. "Vesper: Measuring time-to-interactivity for web pages", *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, 217–231. İnternet adresi: https://www.usenix.org/conference/nsdi18/presentation/netravali-vesper, Son erişim tarihi: 12 Mayıs 2026.

[11] Suryawanshi, P., Dutta, P., L, V., G, D. 2021. "Sustainable and resilience planning for the supply chain of online hyperlocal grocery services", *Sustainable Production and Consumption*, 28, 496–518. DOI: 10.1016/j.spc.2021.05.001.

[12] Urquhart, R., Newing, A., Hood, N., Heppenstall, A. 2022. "Last-mile capacity constraints in online grocery fulfilment in Great Britain", *Journal of Theoretical and Applied Electronic Commerce Research*, 17 (2), 636–651. DOI: 10.3390/jtaer17020033.